Operaciones de Entrada/Salida

La entrada y salida de información (E/S) constituye uno de los pilares fundamentales en el desarrollo de aplicaciones. Python proporciona un conjunto amplio y flexible de mecanismos para interactuar con el usuario, recuperar datos, mostrarlos con claridad y trabajar con distintos formatos y medios de almacenamiento.

1 INTERACCIÓN CON EL USUARIO A TRAVÉS DE LA CONSOLA

La consola es el medio más básico y universal para la interacción entre el usuario y el programa. Python facilita este proceso mediante funciones simples pero potentes, y también ofrece librerías más avanzadas para casos complejos.

1.1 ENTRADA DE DATOS

La función `input()` permite solicitar y recibir información del usuario. Dado que siempre devuelve una cadena, es habitual convertirla al tipo de dato adecuado tras validar su contenido.

```
nombre = input("Introduce tu nombre: ").strip()
edad_str = input("Introduce tu edad: ")

try:
    edad = int(edad_str)
except ValueError:
    print("Error: la edad debe ser un número entero.")
```

!!

Para una interacción más avanzada, como lectura desde flujos estándar o redirección en la línea de comandos, se emplea el módulo sys.

```
import sys
linea = sys.stdin.readline()
```

1.2 SALIDA DE DATOS

La función `print()` es la herramienta principal para mostrar información. Permite configurar separadores, final de línea y el flujo de salida.

```
print("Resultado:", 42, sep=" -> ", end=".\n")
```

!! En el anterior ejemplo, se imprime la cadena `Resultado -> 42.` aunque en el contenido solo se ha especificado `Resultado, 42` puesto que se ha cambiado el elemento de separación de espacio en blanco (`' '`) que es el valor por defecto a (`->`) y el final de la linea se ha cambiado por un `.` acompañado de un salto de línea (`\n`)

También se puede escribir directamente en `stdout` o `stderr`:

```
sys.stdout.write("Mensaje estándar\n")
sys.stderr.write("Mensaje de error\n")
```

1.3 PROCESAMIENTO DE ARGUMENTOS

El procesamiento de argumentos permite que un programa reciba parámetros desde la línea de comandos, por ejemplo:

```
python app.py --archivo datos.csv --verbose
```

USO BÁSICO: SYS.ARGV

Python expone los argumentos en `sys.argv` y recibiendolos como si fuera una lista de cadenas de texto.

!! Todo se interpreta como una cadena de texto y cada parámetro se diferencia de los demás si se deja un espacio entre ellos al llamar al programa.

Si tuvieramos el script de python:

```
import sys

print(sys.argv)
```

Y lo ejecutaramos realizando la siguiente llamada desde la consola:

```
python ejemplo.py uno dos 3
```

La salida resultante sería:

```
['ejemplo.py', 'uno', 'dos', '3']
```

Un ejemplo práctico de como podría ser un programa que reciba por parámetro una cadena de texto para posteriormente saludar a dicho nombre, pero que en caso de que no se pase dicho parámetro salte un mensaje de ayuda:

```python
import sys

if len(sys.argv) < 2:
    print("Uso: python script.py <nombre>")
    sys.exit(1)

nombre = sys.argv[1]
print("Hola,", nombre)
```

ARGPARSE: INTERPRETACIÓN ESTÁNDAR DE ARGUMENTOS EN PYTHON

argparse es una librería que forma de la libería estándar que ofrece:

- ayuda automática
- validación de tipos
- mensajes de error adecuados
- opciones obligatorias y opcionales
- subcomandos
- lectura de argumentos múltiples
- agrupación lógica de opciones

Un ejemplo sencillo de uso de argparse en el cual creamos un argumento posicional (nombre) y un argumento opcional (--edad)

```python
import argparse

parser = argparse.ArgumentParser(description="Ejemplo básico")
parser.add_argument("nombre")     # argumento posicional
parser.add_argument("--edad")     # argumento opcional
args = parser.parse_args()

print(args.nombre, args.edad)
```

La llamada a dicho programa sería del tipo:

```
python app.py Gustave --edad 32

python app.py -h
```

Existen gran varidad de tipos de argumentos y formas de trabajarlos en argparse:

- **Posicionales**: Son obligatorios y se identifican por posición.

```
parser.add_argument("archivo")
```

- **Posicionales**: Se identifican por nombre.

```
parser.add_argument("-o", "--output", help="archivo de salida")
```

- **Flags booleanos**: Valores que almacenan si existen (`True`) o no (`False`)

```
parser.add_argument("-v", "--verbose", action="store_true")
```

- **Conversión a entero (`int`)**:

```
parser.add_argument("--edad", type=int)
```

- **Conversión a `float`**:

```
parser.add_argument("--porcentaje", type=float)
```

- **Conversión a `float` con validación de opciones**:

```
parser.add_argument("--porcentaje", type=float, choices=[0.1, 0.2,
↪  0.3])
```

- **Validación de tipos mediante funciones personalizadas**:

```
def validar_rango(x):
    n = int(x)
    if not (1 <= n <= 10):
        raise argparse.ArgumentTypeError("Debe estar entre 1 y 10")
    return n

parser.add_argument("--nivel", type=validar_rango)
```

- **Parámetros repetibles**:

```
parser.add_argument("--num", action="append", type=int)
```

La llamada a dicho script de Python sería con:

```
python app.py --num 10 --num 20 --num 30
```

Al momento de interpretarlo dentro de `args.num` se almacenará una lista con los valores que pertenezcan a ese parámetro, en este caso `[10, 20, 30]`.

- **Parámetros con repeticiones limitadas**: Son parámetros en los cuales se especifican el número de parámetros que recibe.

```
parser.add_argument("--coords", nargs=2, type=float)
```

```
python py.py --coords 40.40232730664865 -3.706448947518192
```

- **Argumentos obligatorios opcionales**: En ocasiones puede ser que nos sea necesario hacer que un flag sea obligatorio y en caso de que al llamar sin dicho argumento dará error.

```
parser.add_argument("--config", required=True)
```

- **Argumentos mutuamente excluyentes**: Se puede crear un grupo de parámetros y hacer que sean excluyentes entre ellos.

```
group = parser.add_mutually_exclusive_group()
group.add_argument("--quiet", action="store_true")
group.add_argument("--verbose", action="store_true")
```

En este ejemplo, si aparece `--quiet` no puede aparecer `--verbose` y viceversa.

- **Argumentos con valores por defecto**. Se puede hacer que un parámetro tengo un valor por defecto que se establezca en caso de que el usuario no lo establezca como argumento.

```
parser.add_argument("--puerto", type=int, default=8080)
```

También se podría establecer valores que se calculen a partir de una función o una función lambda:

```
parser.add_argument(
    "--fecha",
    default=lambda: datetime.now().isoformat()
)
```

- **Subcomandos**: Se puede crear un argumento que implique que puedan existir otros.

```python
import argparse

parser = argparse.ArgumentParser(prog="gestor")
subparsers = parser.add_subparsers(dest="comando")

# subcomando "add"
cmd_add = subparsers.add_parser("add")
cmd_add.add_argument("nombre")

# subcomando "remove"
cmd_rm = subparsers.add_parser("remove")
cmd_rm.add_argument("id")

args = parser.parse_args()

if args.comando == "add":
    print("Añadiendo:", args.nombre)
elif args.comando == "remove":
    print("Eliminando:", args.id)
```

En este caso, si se usa el argumento `add` se tiene que utilizar el argumento `nombre` y en caso de qeu se use el argumento `remove` se tiene que usar el argumento `id`.

2 PRESENTACIÓN Y FORMATEO DE LA INFORMACIÓN

Es importante mostrar la información de forma clara y lo más legible posible. A continuación se añaden una serie de aspectos que pueden ser interesante que conozcas. Lo más recomendable es el uso de `f-strings`, que permiten incrustar expresiones directamente dentro de cadenas y aplicar formatos complejos. Un concepto interesante es que los datos se pueden alinear:

1. Alineación a izquierda (), derecha (>), centrado (^).

```python
producto = {"nombre": "manzana", "precio": 0.42, total_vendidas:
↪  54398721}
print(f"{producto['nombre']:>10} | {producto['total_vendidas']:^10,} |
↪  {producto['total_vendidas']:>6.2f} €")
```

2. Control del número de decimales:

```
import math
print(f"{math.pi:.2f}" # imprimirá pi con solo dos decimales 3.14
```

3 GESTIÓN DE FECHA Y HORA

En Python, la gestión de la fecha y la hora se hace a través de la librería `datetime`. Para obtener la fecha y la hora actual se utiliza:

```
import datetime

ahora = datetime.datetime.now()
print(ahora)
```

!! Otra alternativa a utilizar `import datetime` es utilizar `from datetime import datetime`, donde especificamos que de la librería `datetime` queremos utilizar el módulo `datetime`. De tal manera que a la hora de especificar la fecha y hora actuales solo tendríamos que decir: `datetime.now()`

3.1 FORMATEAR FECHAS

La función `strftime()` convierte un objeto `datetime` en una cadena siguiendo un patrón.

```
import datetime

ahora = datetime.datetime.now()
formato = ahora.strftime("%Y-%m-%d %H:%M:%S")
print(formato)
```

Código	Significado	Ejemplo
%Y	Año (4 dígitos)	2025
%y	Año (2 dígitos)	25
%m	Mes (01–12)	12
%B	Nombre del mes	December
%d	Día (01–31)	11
%A	Día de la semana	Thursday
%H	Hora (00–23)	17
%I	Hora (01–12)	05
%M	Minutos	45
%S	Segundos	32
%f	Microsegundos	123456
%z	Zona horaria	+0100
%Z	Nombre de la zona	CET

Puedes configurar las fechas con idioma específico si utilizas la librería `locale`. Por ejemplo, si especificas `es_ES.UTF-8` le estás especificando que ponga la fecha con los días y los meses en castellano.

```
import datetime
import locale

locale.setlocale(locale.LC_TIME, 'es_ES.UTF-8')

ahora = datetime.datetime.now()
print(ahora.strftime("%A %d de %B de %Y"))
```

!! El estándar de fechas más utilizado es ISO 8601, puedes especificar que utilice dicho estándar con el método `isoformat()` sobre un objeto de tipo `datetime`. Un ejemplo de fecha en ISO 8601 es `2025-12-11T17:45:52.123456`.

3.2 PARSEAR FECHAS DESDE CADENAS DE TEXTO

En muchas ocasiones no solo vas a tener que crear una fecha, sino que tendrás que leerla y pasarla a un objeto de tipo de `datetime`, para ello se recomienda el uso de `strptime()` donde le especificarás por parámetro el formato de la cadena.

```
cadena = "11/12/2025 17:45"
fecha = datetime.datetime.strptime(cadena, "%d/%m/%Y %H:%M")
```

4 LECTURA Y ESCRITURA DE FICHEROS

Python no se limita a la consola: integra un amplio conjunto de herramientas para trabajar con datos en distintos formatos y fuentes. El manejo de ficheros se realiza mediante:

- **Rutas (`file`)** ya sean absolutas (desde el directorio raíz hasta donde está el directorio o fichero que queremos utilizar) o relativas (desde donde se ejecuta el fichero hasta donde está el directorio o fichero que queremos utilizar).
- **Modos de apertura (`mode`)**:
  - `r`: abre el fichero en modo lectura
  - `w`: abre el fichero en modo escritura. Recuerda que si se abre en modo escritura, se sobrescribe el contenido.
  - `a`: abre el fichero en modo agregar. Es un modo especial de escritura en el que no se sobrescribe el fichero, sino que se añade al final del mismo el contenido que escribamos.
- **Codificación (`encoding`)** El formato en el que se va a leer o escribir el fichero: normalmente utilizaremos `utf-8` dado que es el formato que permite el uso de tildes y ñ.

4.1 ACCESO A FICHEROS

Para acceder a ficheros utilizaremos la función `open`.

!!

Recuerda cerrar las conexiones o los ficheros abiertos una vez que te hayas conectado. Un fichero abierto

- Consume un descriptor de fichero, memoria y buffers de E/S. Los sistemas operaativos tienen un limite de archivso abiertos y no cerrarlos puede causar errores.
- En algunos sistemas, un archivo abierto puede quedar bloqueado para otros procesos. Si no se cierra, otros programas o incluso tu propio script no podrán acceder a él.
- Cuando se escribe en un archivo, los datos suelen quedarse en un buffer en memoria. Si no se cierra parte de la información puede no escribirse o el archivo puede quedar corrupto o incompleto.
- Un fichero mal gestionado puede generar pérdidas de memoria.

Todo fichero abierto debe de ser cerrado, esto se puede gestionar de dos maneras:

- **Cierre manual del archivo**:

```
try:
    f = open("ficherillo")
    procesar(f)
finally:
    f.close()
```

- **Cierre automático del archivo**:

```
with open("ficherillo") as f:
    procesar(f)    # Si aquí falla, igualmente se cierra el archivo
```

!! `with` garantiza la adquisición del recurso al entrar en el bloque(`__enter__`) y la liberación al salir(`__exit__`).

LEER FICHEROS

!! **Recuerda:** Para poder leer un fichero es necesario que exista, si no se lanzará un `FileNotFoundError`.

1. **Leer todo el fichero**

```
with open("datos.txt", "r", encoding="utf-8") as f:
    contenido = f.read()
```

2. **Leer el fichero línea a línea**

```
with open("datos.txt", "r", encoding="utf-8") as f:
    linea = f.readline()
    while linea:
        linea = f.readline()
```

3. **Leer todo el fichero y almacenar cada línea en una lista para usarlo más tarde**

```
with open("datos.txt", "r", encoding="utf-8") as f:
    lineas = f.readlines()

for linea in lineas:
    print(linea)
```

ESCRIBIR FICHEROS

!! Cuando se escribe en ficheros, se escribe lo que le mandas escribir, por lo tanto, si no le especificas que escriba un salto de línea con \n todo lo escrito estará en la misma línea.

1. **Escribir ficheros a partir de una cadena de texto**

```
with open("ficherillo.txt", "w", encoding="utf-8") as f:
    f.write("linea 1\n")
```

2. **Escribir ficheros a partir de una lista**

```
lineas = [f"linea {i}\n" for i in range(10)]
with open("ficherillo.txt", "w", encoding="utf-8") as f:
    f.writelines(lineas)
```

4.2 MODOS DE APERTURA DE FICHEROS HABITUALES

Uso	Modo
Leer texto	r
Escribir texto	w
Añadir texto	a
Leer binario	rb
Escribir binario	wb
Leer y escribir	r+

4.3 FICHEROS JSON

Un fichero JSON (JavaScript Object Notation) es un archivo de texto plano que almacena información estructurada siguiendo una sintaxis estándar, ligera y legible tanto por personas como por máquinas. Se utiliza ampliamente para el intercambio de datos, la configuración de aplicaciones y la comunicación entre sistemas, especialmente en entornos web y APIs.

Aunque es texto plano, JSON organiza la información de forma jerárquica mediante:

- Objetos → pares clave–valor ({})
- Arrays → listas ordenadas ([])

La librería `json` forma parte de la biblioteca estándar de Python y proporciona herramientas para serializar y deserializar datos en formato JSON.

SERIALIZAR

Serializar supone convertir objetos Python a JSON.

- Podemos serializar para transformar un diccionario a una cadena de texto en formato JSON.

```
import json
datos = {"nombre": "Gustave", "edad": 32,"activo": True}
json_str = json.dumps(datos)
print(json_str)
```

- También podemos serializar para transformar un diccionario a una cadena de texto en formato json y posteriormente escribirlo en un fichero.

```
import json
datos = {"nombre": "Gustave", "edad": 32,"activo": True}
with open("datos.json", "w", encoding="utf-8") as f:
    json.dump(datos, f, indent=4)
```

!! El parámetro `indent` permite que se añada tabulaciones para diferenciar niveles dentro del dicc... el fichero `json`.

DESERIALIZAR

Deserializar supone convertir un texto en formato JSON a tipos de Python, normalmente a un diccionario.

```
import json
with open("datos.json", "r", encoding="utf-8") as f:
    datos = json.load(f)
```

MANEJO DE ERRORES

```
import json
try:
    with open("datos.json", "r", encoding="utf-8") as f:
        datos = json.load(f)
except json.JSONDecodeError:
    print("JSON mal formado")
except FileNotFoundError:
    print("Archivo no encontrado")
```

4.4 FICHEROS CSV

Un fichero CSV (Comma-Separated Values) es un archivo de texto plano que almacena información estructurada siguiente lógica:

- La primera línea habitualmente corresponde a las cabeceras donde se describe que va en cada columna.
- Cada línea corresponde a un registro.
- Los campos están separados por un delimitador. Habitualmente coma (`,`) o punto y coma (`;`), pero pueden usarse otros.
- No contiene información de tipos: todo se interpreta como texto.

LECTURA DE ARCHIVOS CSV

A continuación, se muestra un ejemplo de lectura básica de un fichero CSV, cada fila leída por el `lector` será interpretada como un objeto de tipo `list` compuesto por cadenas de texto con cada uno de los valores leídos en dicha línea.

```
import csv
with open("datos.csv", "r", encoding="utf-8", newline="") as f:
    lector = csv.reader(f)
    for fila in lector:
        print(fila)
```

El anterior código va a leer todas las filas del fichero `datos.csv`, si por un casual quisiéramos evitar que lea la primera fila, puesto que habitualmente va a ser la de las cabeceras podríamos hacerlo con el método `next()`:

```
import csv
with open("datos.csv", "r", encoding="utf-8", newline="") as f:
    lector = csv.reader(f, delimiter=",")
    cabeceras = next(lector)  # Salta la primera fila
    print(f"cabeceras: {cabeceras}")
    for fila in lector:
        print(fila)
```

Si por un casual se deseara interpretar cada línea como diccionario, tomando como clave el nombre de la columna en la cabecera y como valor lo establecido para dicha línea en dicha columna se puede utilizar la clase `csv.DictReader()`

```
import csv
with open("datos.csv", newline="", encoding="utf-8") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila)
        print(f"nombre: {fila['nombre']}")
```

ESCRITURA DE FICHEROS CSV

El siguiente fragmento crea un fichero CSV `salida.csv` con dos líneas, la primera con las cabeceras y la segunda con datos.

```
import csv
with open("salida.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.writer(f)
    escritor.writerow(["id", "nombre", "edad"])
    escritor.writerow([1, "Lune", 32])
```

El problema del anterior fragmento es que cada vez que lo ejecutemos sobrescribirá el fichero, si quisiéramos añadir líneas al fichero deberíamos no escribir las cabeceras y abrirlo en modo agregación.

```python
import csv
with open("salida.csv", "a", newline="", encoding="utf-8") as f:
    escritor = csv.writer(f)
    escritor.writerow([2, "Gustave", 32])
```

Otra alternativa a la escritura que se muestra en el ejemplo anterior, sería escribir con claves haciendo uso de la clase `csv.DictWriter()` a la cual le pasaremos una lista con los campos de la cabecera.

```python
import csv
campos = ["id", "nombre", "edad"]
with open("salida.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.DictWriter(f, fieldnames=campos)
    escritor.writeheader()
    escritor.writerow({"id": 1, "nombre": "Lune", "edad": 32})
```

!! Recuerda, para no sobrescribir utiliza `mode="a"` y no escribas la linea de cabeceras. En este último ejemplo saltándote `escritor.writeheader()`

PARÁMETROS IMPORTANTES

Parámetro	Uso
delimiter	Separador (',', ';', '\t')
quotechar	Carácter de cita
quoting	Control de comillas
lineterminator	Fin de línea
newline	Debe ser "" al abrir

4.5 SERIALIZACIÓN CON `PICKLE`

La librería `pickle` forma parte de la biblioteca estándar de Python y se utiliza para serializar y deserializar objetos Python, es decir, convertir estructuras de datos y objetos complejos en una secuencia de bytes para poder almacenarlos o transmitirlos, y posteriormente reconstruirlos.
`pickle` implementa un mecanismo de serialización nativa de Python:

- Objeto Python a bytes (pickling)
- Bytes a objeto Python (unpickling)

!! A diferencia de formatos como JSON o CSV, `pickle` puede manejar clases, instancias, funciones o estructuras complejas. Es recomendable para la persistencia rápida de objetos o cachés locales, pero no es recomendado para intercambio entre lenguaje o datos externos.

SERIALIZACIÓN CON PICKLE

- **Guardar un objeto en un archivo**. Para guardar un objeto en un archivo el modo binario ("wb") es obligatorio.

```
import pickle
datos = {"usuario": "user", "roles": ["admin", "editor"]}
with open("datos.pkl", "wb") as f:
    pickle.dump(datos, f)
```

- **Cargar un objeto desde un archivo**

```
import pickle
with open("datos.pkl", "rb") as f:
    datos = pickle.load(f)
print(datos["usuario"])
```

- **`dumps()` y `loads()`**. Con estos métodos puedes cargar en memoria caché o transmitir los objetos que serialices con pickle sin la necesidad de utilizar ficheros.

```
import pickle
blob = pickle.dumps(datos)
objeto = pickle.loads(blob)
```

5 MÉTODOS DE ACCESO A CONTENIDOS EN FICHEROS

Los métodos de acceso al contenido de ficheros determinan cómo se leen o escriben los datos, en qué orden y con qué nivel de control sobre el flujo de información.

5.1 ACCESO SECUENCIAL

Es el método más común. Los datos se procesan en el orden en que aparecen en el archivo, desde el inicio hasta el final. Se caracteriza por:

- No permite saltos arbitrarios.

- Ideal para archivos de texto y procesamiento línea a línea.
- Bajo consumo de memoria.

```
with open("datos.txt", "r", encoding="utf-8") as f:
    for linea in f:
        print(linea.strip())
```

5.2 ACCESO DIRECTO O ALEATORIO

Permite posicionarse en cualquier parte del fichero usando desplazamientos (offsets), sin leer el contenido anterior. Este tipo de acceso tiene las siguientes características:

- Requiere conocimiento de la estructura del archivo.
- Muy eficiente en archivos binarios.
- Acceso inmediato a posiciones concretas.

```
with open("datos.txt", "rb") as f:
    f.seek(10)
    bloque = f.read(20)
```

!!

- **seek(posición)**: mueve el puntero
- **tell()**: devuelve la posición actual

5.3 ACCESO POR BLOQUES

El fichero se lee o escribe en fragmentos de tamaño fijo, en lugar de línea a línea o completo. Especialmente útil para comunicación por red o archivos grandes.

```
with open("datos.bin", "rb") as f:
    while True:
        bloque = f.read(1024)
        if not bloque:
            break
        procesar(bloque)
```

5.4 MEMORIA MAPEADA

La memoria mapeada (memory-mapped files) es un método avanzado de acceso a ficheros que permite tratar el contenido de un archivo como si fuera un bloque de memoria RAM, sin necesidad

de leerlo explícitamente mediante llamadas tradicionales de entrada/salida. Se utiliza mediante el módulo `mmap`.

La memoria mapeada establece una asociación directa entre un fichero y un espacio de direcciones de memoria del proceso:

- El sistema operativo carga en memoria solo las partes necesarias del archivo.
- El acceso se realiza mediante lecturas y escrituras en memoria, no con `read()` o `write()`.
- El sistema operativo gestiona la paginación automáticamente.

Este método de acceso es especialmente útil cuando se tiene que procesar archivos de gran tamaño, en motores de búsqueda o en la comunicación entre procesos.

EJEMPLO DE LECTURA

```
import mmap
with open("datos.txt", "r+b") as f:
    with mmap.mmap(f.fileno(), 0) as mm:
        print(mm[:20])
```

- 0 indica mapear el archivo completo
- `fileno()` obtiene el descriptor del fichero

ACCESO POR ÍNDICE

```
import mmap
with open("datos.txt", "r+b") as f:
    with mmap.mmap(f.fileno(), 0) as mm:
        print(mm[10])      # byte en la posición 10
        print(mm[10:20])   # rango de bytes
```

BÚSQUEDA EFICIENTE EN ARCHIVOS GRANDES

```
import mmap
with open("datos.txt", "r+b") as f:
    with mmap.mmap(f.fileno(), 0) as mm:
        pos = mm.find(b"ERROR")
        if pos != -1:
            print(f"Encontrado en: {pos}")
```

ESCRITURA CON MEMORIA MAPEADA

```
import mmap
with open("datos.bin", "r+b") as f:
    with mmap.mmap(f.fileno(), 0) as mm:
        mm[0:4] = b"TEST"
```

ESCRITURA CON MEMORIA MAPEADA

```
import mmap
with open("datos.bin", "r+b") as f:
    with mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ) as mm:
    print(mm[:50])
```